GC-F-004v.09

COLOMBIA POTENCIA DE LA VIDA

SENA

Análisis y desarrollo de software.

Instructor

Ing. Diego Fernando Calderon Silva
Correo: dfcalderon@sena.edu.co

React + Ts

¿Qué es React?

React es una biblioteca de JavaScript de código abierto utilizada para construir interfaces de usuario, especialmente para aplicaciones web de una sola página. Fue desarrollada por Facebook (ahora Meta) y lanzada en 2013.



Creando un proyecto

Para crear un proyecto en vite, y luego tomar react, usamos

```
elingelindo@iMac-de-Diego reactdecero % npm create vite@latest
✔ Project name: … react-app
✔ Select a framework: › React
✔ Select a variant: › TypeScript

Scaffolding project in /Users/elingelindo/Library/CloudStorage/C

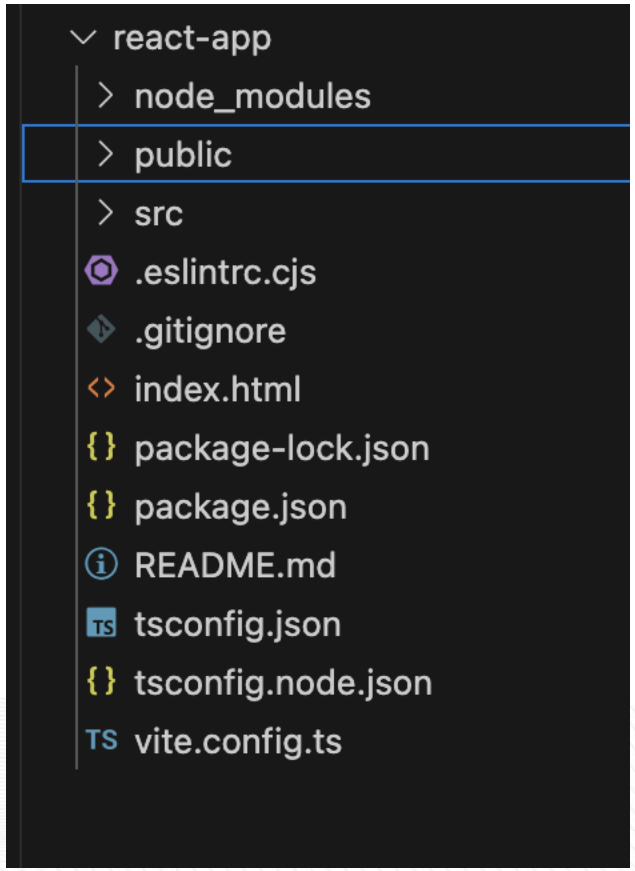
Done. Now run:

  cd react-app
  npm install
  npm run dev
```

Ahora ingresaremos a nuestro proyecto con cd, instalaremos las dependencias y finalmente ejecutaremos nuestro proyecto

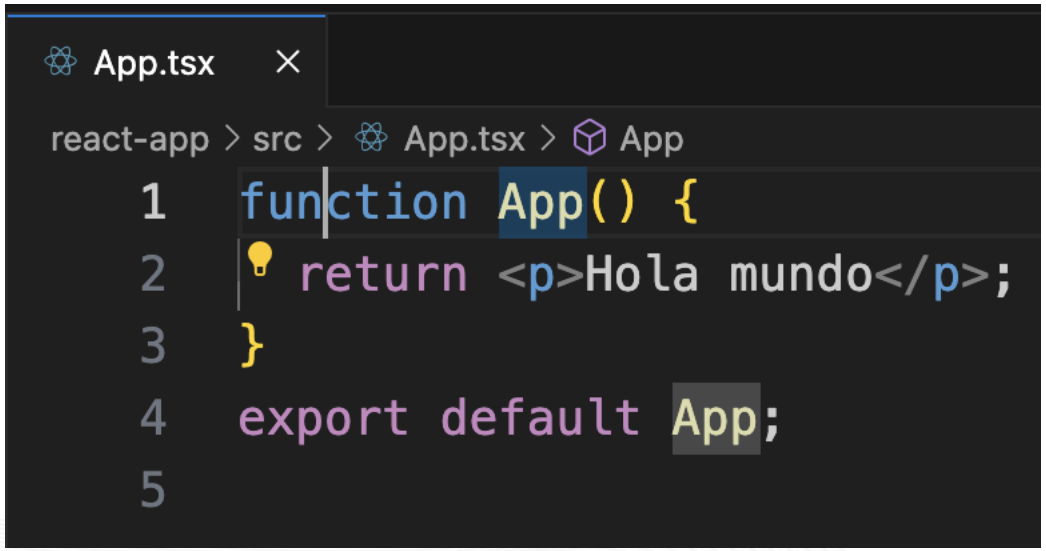
Estructura del proyecto

Analicemos un poco.



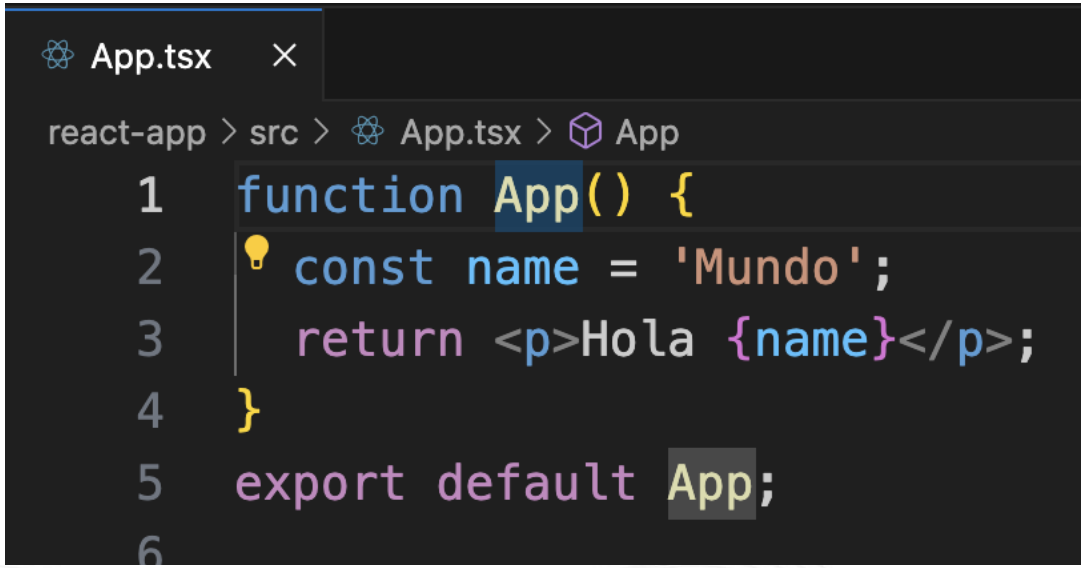
Componentes

Empecemos por crear nuestros propios componentes, para esto eliminemos App.tsx y creemos una función llamada App la cual exportaremos por defecto y retornaremos únicamente una etiqueta <p>



Componentes

Si bien podemos retornar etiquetas html, también podemos retornar variables o constantes creadas dentro de la función App, por eje:



Componentes

Con esto, podremos tener un control total de lo que estamos haciendo llevando al punto de generar condicionales así:

App.tsx

react-app > src > App.tsx > App

```
1  function App() {
2    const name = 'Mundo';
3    if(name){
4      return <p>Hola, vas a retornar el positivo de {name}</p>;
5    }
6    return <p>Hola Mundo</p>;
7  }
8  export default App;
9
```

Claramente acabamos de crear un componente condicional, que funciona, pero no está escrito de la mejor forma, es por esto que vamos a generar una estructura de trabajo para los componentes.

Componentes

Vamos a crear un archivo llamado Titulo.tsx en la raíz de src y pegamos lo que acabamos de crear en App.tsx, solo que cambiando el nombre de la función de App a Título.

App.tsx | Titulo.tsx

react-app > src > Titulo.tsx > ...

```tsx
function Titulo() {
  const name = "Mundo";
  if (name) {
    return <p>Hola, vas a retornar el positivo de {name}</p>;
  }
  return <p>Hola Mundo</p>;
}
export default Titulo;
```

Ahora, en App.tsx eliminamos el contenido de la función y retornamos Titulo asi:

App.tsx | Titulo.tsx

react-app > src > App.tsx > ...

```tsx
import Titulo from "./Titulo";

function App() {
  return <Titulo />
}
export default App;
```

Componentes

Vamos a crear un archivo llamado Titulo.tsx en la raíz de src y pegamos lo que acabamos de crear en App.tsx, solo que cambiando el nombre de la función de App a Título.

App.tsx | Titulo.tsx

react-app > src > Titulo.tsx > ...

```tsx
function Titulo() {
  const name = "Mundo";
  if (name) {
    return <p>Hola, vas a retornar el positivo de {name}</p>;
  }
  return <p>Hola Mundo</p>;
}
export default Titulo;
```

Ahora, en App.tsx eliminamos el contenido de la función y retornamos Titulo asi:

App.tsx | Titulo.tsx

react-app > src > App.tsx > ...

```tsx
import Titulo from "./Titulo";

function App() {
  return <Titulo />
}
export default App;
```

Componentes

Instalemos Bootstrap ya que es un framework de css que nos ayuda a que los componentes se vean mejor, para instalar usemos el comando: `npm i bootstrap`

Como vamos a usar un framework, eliminemos lo estilos que vienen por defecto configurados en react ya que no los usaremos.

```
},
"dependencies": {
  "bootstrap": "^5.3.3",
  "react": "^18.2.0",
  "react-dom": "^18.2.0"
},
```

Importemos Bootstrap en main.tsx asi:

```
import 'bootstrap/dist/css/bootstrap.min.css'
```

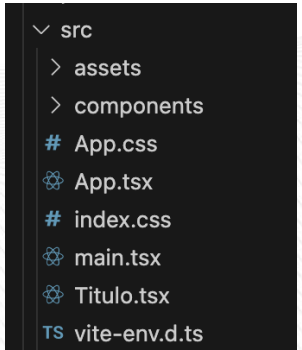
Component card

Vamos a crear el primer componente basados en la documentación de Bootstrap, para eso vamos a la documentación oficial, y busquemos la sección de “ card ”. Busquemos una que no contenga imágenes.

HTML

```html
<div class="card" style="width: 18rem;">
  <img src="..." class="card-img-top" alt="...">
  <div class="card-body">
    <h5 class="card-title">Card title</h5>
    <p class="card-text">Some quick example text to build on the card title and make up
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
</div>
```

Ahora, crearemos nuestro componente, pero antes, debemos crear un directorio llamado components, en la raíz de src.



Component card

Una vez dentro, crearemos un archivo llamado Card.tsx, pegare el código de la docu de Bootstrap, y lo modificare para que react lo entienda asi:
Lo que diga class, lo cambiare por className, lo que diga for, lo cambiare por htmlFor.

App.tsx 1 | main.tsx | Card.tsx | Titulo.tsx

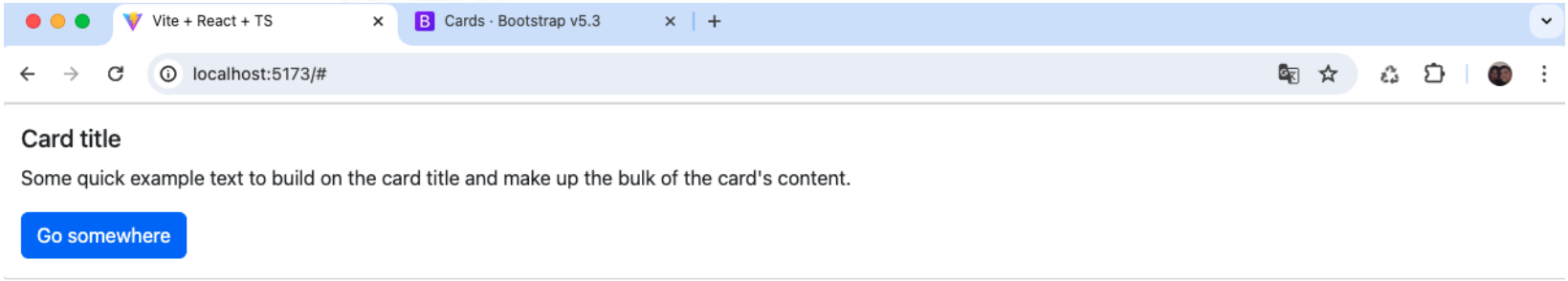
react-app > src > components > Card.tsx > ...

```tsx
function Card() {
  return (
    <div className="card">
      <div className="card-body">
        <h5 className="card-title">Card title</h5>
        <p className="card-text">
          Some quick example text to build on the card title and make up the
          bulk of the card's content.
        </p>
        <a href="#" className="btn btn-primary">
          Go somewhere
        </a>
      </div>
    </div>
  );
}

export default Card;
```

Component card

Se debería ver algo como esto:



Ahora, demosle un poco mas de estilo, vamos a darle un ancho fijo en Card.tsx

```
<div className="card" style="">
```

Como podemos observar, si le pasamos un string, nos mostrara un error, ya que react espera un objeto asi que lo debemos pasar así:

```
<div className="card" style={width}>
```

Pero debemos crear el objeto como una constante asi:

Component card

Pero debemos crear el objeto como una constante así:

App.tsx 1 | main.tsx | Card.tsx | Titulo.tsx

react-app > src > components > Card.tsx > Cerrar (⌘W)

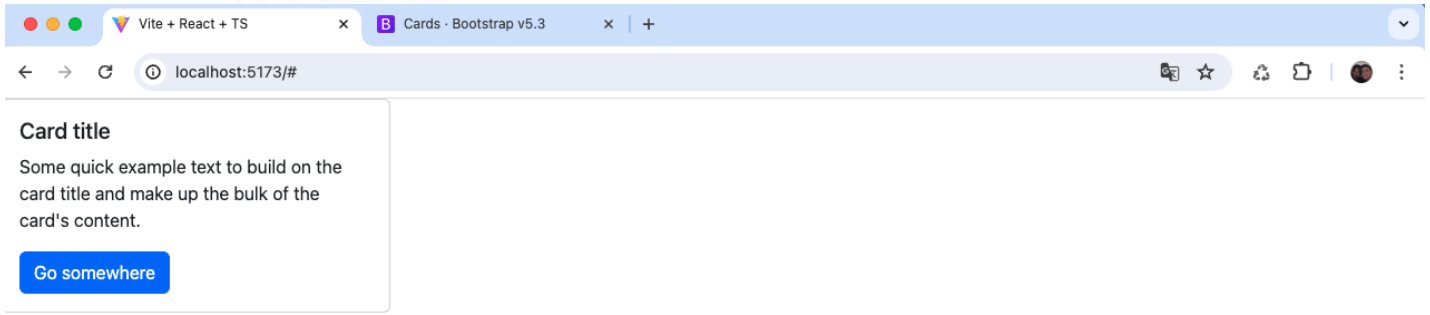
```tsx
function Card() {
    const width = {
        width: "350px",
    };
    return (

    <div className="card" style={width}>
      <div className="card-body">
        <h5 className="card-title">Card title</h5>
        <p className="card-text">
          Some quick example text to build on the card title and make up the
          bulk of the card's content.
        </p>
        <a href="#" className="btn btn-primary">
          Go somewhere
        </a>
      </div>
    </div>
  );
}

export default Card;
```

Component card

Y desde el navegador deberíamos tener algo así:



Fragment

Hasta ahora nuestro componente de Card se encuentra con un solo estilo, y dijimos que esto era porque debía recibir un objeto y no un string, así que solo para fines prácticos, vamos a eliminar la constante y vamos a indicarle a reac un pequeño hack, para generar mayor entendimiento del tema.

Esta no es la mejor practica del mundo, pero quiero que etiendan que los objetos se usan de esta manera también, las primeras llaves lo reciben como parámetro, y las segundas indican que es un objeto. Por lo que desde el navegador, debemos estar viendo lo mismo que hace un moneto.

react-app > src > components > Card.tsx > ...

```
1  function Card() {
2    return (
3      <div
4        className="card"
5        style={{
6          width: "350px",
7        }}
8      >
```

Fragment

Hasta ahora nuestro componente de Card se encuentra con absolutamente todas sus partes escritas en duro dentro del mismo componente, sin embargo, con el objetivo de poder mantener y reutilizar el código vamos a empezar a separar este componente en otros más pequeños, ejemplo, separemos el titulo para poder pasarle h1, h2, h3 o el elemento que deseemos, y también, pasemos un cuerpo de manera dinámica. Por lo que tomare todo el cuerpo y lo sacare del componente así:

react-app > src > components > Card.tsx > Card

```
1  function Card() {
2    return (
3      <div
4        className="card"
5        style={{
6          width: "350px",
7        }}
8      >
9        <div className="card-body"></div>
10     </div>
11   );
12 }
13
14 export default Card;
```

Fragment

Ahora, debajo del componente Card, creare otro llamado CardBody el cual contendrá el cuerpo asi:

react-app > src > components > Card.tsx > ...

```tsx
function Card() {
  return (
    <div
      className="card"
      style={{
        width: "350px",
      }}
    >
      <div className="card-body">
        Hola mundo
      </div>
    </div>
  );
}

export function CardBody(){
    return(
        <h5 className="card-title">Card title</h5>
        <p className="card-text">
          Some quick example text to build on the card title and make up the
          bulk of the card's content.
        </p>
        <a href="#" className="btn btn-primary">
          Go somewhere
        </a>
    );
}

export default Card;
```

Fragment

Lo primero que vemos, es un error, así que, para solucionarlo, retornaremos todo lo que este dentro de un div

```
export function CardBody() {
  return (
    <div>
      <h5 className="card-title">Card title</h5>
      <p className="card-text">
        Some quick example text to build on the card title and make up the bulk
        of the card's content.
      </p>
      <a href="#" className="btn btn-primary">
        Go somewhere
      </a>
    </div>
  );
}
```

Esto nos soluciona el problema, pero nos agrega otro y es que añade un div de más al proyecto que es innecesario, asi que para esto usaremos los fragments.

Fragment

Para usarlo, empecemos por importarlo así:

```
import { Fragment } from "react";
```

Y para usarlo, reemplazamos el div, por Fragment así:

```
export function CardBody() {
  return (
    <Fragment>
      <h5 className="card-title">Card title</h5>
      <p className="card-text">
        Some quick example text to build on the card title and make up the bulk
        of the card's content.
      </p>
      <a href="#" className="btn btn-primary">
        Go somewhere
      </a>
    </Fragment>
  );
}
```

Fragment

Esta es la forma como react nos enseña a hacerlo, pero la forma más rápido de hacerlo, es directamente quitando el texto de la etiqueta y dejándola libre, react entenderá que esto, lo que busca es quitar el div generado de más y que es un fragment del proyecto.

```
<>
  <h5 className="card-title">Card title</h5>
  <p className="card-text">
    Some quick example text to build on the card title and make up the bulk
    of the card's content.
  </p>
  <a href="#" className="btn btn-primary">
    Go somewhere
  </a>
</>
```

Fragment

Ahora lo que haremos, será pasar el componente CardBody, a Card así:

Tal como vemos, estamos usando todos los componentes en un solo archivo, pero y entonces ¿Cómo podemos reutilizarlos?

Justamente para eso usaremos propiedades.

App.tsx 1 | main.tsx | Card.tsx | Titulo.tsx

react-app > src > components > Card.tsx > ...

```tsx
function Card() {
  return (
    <div
      className="card"
      style={{
        width: "350px",
      }}
    >
      <div className="card-body">
        <CardBody />
      </div>
    </div>
  );
}

export function CardBody() {
  return (
    <>
      <h5 className="card-title">Card title</h5>
      <p className="card-text">
        Some quick example text to build on the card title and make up the bulk
        of the card's content.
      </p>
      <a href="#" className="btn btn-primary">
        Go somewhere
      </a>
    </>
  );
}

export default Card;
```

Pasando props

En este momento tenemos dos componentes, Card y CardBody, por lo que vamos a sacar a CarBody de Card y vamos a reemplazar únicamente por un Hola mundo.

react-app > src > components > Card.tsx > ...

```tsx
function Card() {
  return (
    <div
      className="card"
      style={{
        width: "350px",
      }}
    >
      <div className="card-body">
        Hola mundo
      </div>
    </div>
  );
}
```

Pasando props

Ahora, ¿cómo podemos hacer uso de las propiedades?

Lo primero que debemos hacer es pasarle una propiedad cualquiera al componente

react-app > src > App.tsx > ...

```tsx
import Card from "./components/Card";

function App() {
  return <Card body={"Hola mundo"} />;
}
export default App;
```

Como podemos ver, la propiedad body se le está enviando al componente Card y contiene un string con el valor “Hola mundo”; en este momento podemos ver que hay un error así que vamos a eliminar el error.

Pasando props

Para eliminar el error, debemos ir a la definición de la función Card, y dentro de los paréntesis le vamos a decir que recibe “props” y después, dentro de las llaves, cuando queremos obtener todas las propiedades de este componente vamos a hacer un object destructuring así:

```
function Card(props) {

    const { body } = props;
```

Y para usarlo, lo único que deberíamos hacer es reemplazar el Hola mundo, por la propiedad asi:

```
1  function Card(props) {
2    const { body } = props;
3
4    return (
5      <div
6        className="card"
7        style={{
8          width: "350px",
9        }}
10     >
11       <div className="card-body">{body}</div>
12     </div>
13   );
14 }
```

Pasando props

Pero si observamos, sigue mostrando error a la hora de recibir props en Card y este error nos indica que props contiene any implícitamente, y esto es porque cada que usamos un componente debemos indicarle cuales son las propiedades que este recibe, en este caso un body de tipo string. Asi que tendremos que definir una interface y luego el nombre, esto antes de la función Card.

```
interface Props {
    body: string;
}

function Card(props: Props) {
  const { body } = props;
```

Como se observa, se construye la interface, Props, y este mismo tipo se le pasa a la propiedad props.

Multiples props

Hasta ahora solo hemos pasado body, pero veamos como pasar múltiples props, lo primero es ir hasta CardBody y vemos que, en el h5, el componente necesita un título y además necesita un texto. La etiqueta <a> no nos interesa por lo que la eliminamos, pero vamos a hacer que el componente CardBody, reciba el titulo y el contenido de la etiqueta <p>

```
export function CardBody() {
  return (
    <>
      <h5 className="card-title">Card title</h5>
      <p className="card-text">
        Some quick example text to build on the card title and make up the bulk
        of the card's content.
      </p>
      <a href="#" className="btn btn-primary">
        Go somewhere
      </a>
    </>
  );
}
```

Multiples props

Lo primero es indicarle a CardBody que va a recibir unas props, del tipo CardBodyProps, la cual recibirá un title tipo string, y un text tipo string, para luego hacer el destructuring. Todo junto se vería así:

```
interface CardBodyProps {
  title: string;
  text: string;
}

export function CardBody(props: CardBodyProps) {
    const { title, text } = props;
  return (
```

El error que muestra es porque no estamos usando las propiedades por lo que les daremos uso así:

```
<h5 className="card-title">{title}</h5>
<p className="card-text">
  {text}
</p>
```

Multiples props

Ahora, vamos a usar CardBody, en lugar de Card, así que vamos a App.tsx y reemplazamos Card.

```tsx
import {CardBody} from "./components/Card";

function App() {
  return <CardBody body={"Hola mundo desde app.tsx"} />;
}
export default App;
```

Vemos que hay un error y es porque el tipo body, no es una propiedad asignable, pero sabemos, que debemos asignar title y text.

```tsx
function App() {
  return <CardBody title={"Hola mundo"} text={"Hola mundo desde app.tsx"} />;
}
```

También podemos decirle a react, que las props pueden ser opcionales haciendo esto:

```tsx
interface CardBodyProps {
  title?: string;
  text?: string;
}
```

Children

Finalmente hemos usado multriples props, pero lo estamos usando desde CardBody y no desde Card, como lo pensamos inicialmente asi que debemos restructurar la forma como venimos usando. Para esto, vamos a sacar CardBody de App.tsx y dejamos Card junto con su propiedad.

react-app > src > App.tsx > ...

```tsx
import Card, {CardBody} from "./components/Card";

function App() {
  return <Card body={"Hola mundo"}/>;
}
export default App;
```

Sin embargo, esta no es la única forma que tenemos para pasarle propiedades al componente Card, existe otra forma que es con la propiedad “ children ”, así que vamos a eliminar la propiedad y le pasamos un texto de Hola mundo así:

```tsx
function App() {
  return <Card>Hola mundo</Card>;
}
```

Pero también debemos cambiar la propiedad body, dentro de Card

Children

El cambio que vamos a hacer es reemplazar body, por children así.

react-app > src > components > Card.tsx > CardBodyProps > text

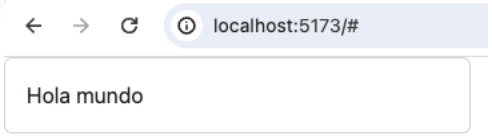
```tsx
interface Props {
    children: string;
}

function Card(props: Props) {
  const { children } = props;

  return (
    <div
      className="card"
      style={{
        width: "350px",
      }}
    >
      <div className="card-body">{children}</div>
    </div>
  );
}
```

Children

Si vemos en el navegador, todo sigue funcionando, ¿pero ¿qué pasa si quiero agregarle algo mucho más estructurado?



Por ejemplo, intentemos pasarle otro componente:

```
function App() {
  return (
    <Card>
      <CardBody />
    </Card>
  );
```

Como vemos, nos da error y es porque children, espera que sea un string, pero claramente no le estamos pasando un string, le estamos pasando un componente CardBody.

Children

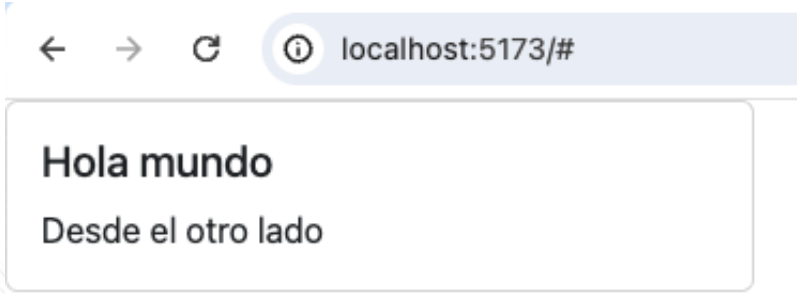
Para corregir este problema, vamos a decirle que no reciba un string en la interface, sino ReactNode:

```tsx
import { ReactNode } from "react";

interface Props {
  children: ReactNode;
}
```

Con esta modificación, ya podremos pasarle un componente junto con sus propiedades.

```tsx
function App() {
  return (
    <Card>
      <CardBody  title="Hola mundo" text="Desde el otro lado"/>
    </Card>
  );
}
```



Imprimir listas

Hemos visto como desacoplar componentes y cómo podemos pasar diferentes y múltiples propiedades, así que es hora de ver como imprimir listas. Vamos a boostrap y buscamos el ejemplo básico de list group y vamos a crear el componente List, el cual se va a pasar dentro de Card.
Me gusta empezar llamando el componente en App.tsx para luego ir a crearlo.

react-app > src > App.tsx > ...

```tsx
import Card, { CardBody } from "./components/Card";
import List from "./components/List";

function App() {
  return (
    <Card>
      <CardBody  title="Hola mundo" text="Desde el otro lado"/>
      <List/>
    </Card>
  );
}
export default App;
```

Imprimir listas

Luego de crear el componente debería verse así:

react-app > src > components > List.tsx > ...

```tsx
type Props = {};

function List({}: Props) {
  return (
    <ul className="list-group">
      <li className="list-group-item">An item</li>
      <li className="list-group-item">A second item</li>
      <li className="list-group-item">A third item</li>
      <li className="list-group-item">A fourth item</li>
      <li className="list-group-item">And a fifth one</li>
    </ul>
  );
}

export default List;
```

¿Qué hay diferente el componente?

Imprimir listas

Con lo que acabamos de hacer tenemos una lista estática, que si bien nos sirve para visualizar como se crea no es útil en el mundo de la codificación, es por esto que la vamos a hacer dinámica. Primero le vamos a dar propiedades:

```
type Props = {
    data: string[];
};
```

Primero le pasamos una prop, llamada data, la cual definimos como un arreglo de strings, pero al guardar, vemos que App.tsx, no da error.



Imprimir listas

Así que debemos enviarle data, teniendo en cuenta que es un arreglo de strings:

```
function App() {
  const list = [
    "Goku",
    "Vegeta",
    "Trunks",
    "Gohan",
  ]
  return (
    <Card>
      <CardBody  title="Hola mundo" text="Desde el otro lado"/>
      <List data={list}/>
    </Card>
  );
```

Imprimir listas

Ahora, lo recibimos en el componente y hacemos un destructoring, para no tener problemas. Recordemos que es un array, y que necesitamos que la lista sea dinámica, por lo que como primera medida se van a eliminar todas las etiquetas <li> y se deja una, para después usar el método .map el cual nos permitirá recorrer cada campo y escribirlo

react-app > src > components > List.tsx > default

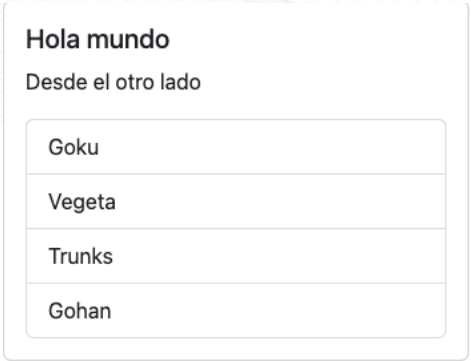
```tsx
type Props = {
  data: string[];
};

function List({ data }: Props) {
  return (
    <ul className="list-group">
      {data.map((elemento) => (
        <li className="list-group-item">{elemento}</li>
      ))}
    </ul>
  );
}

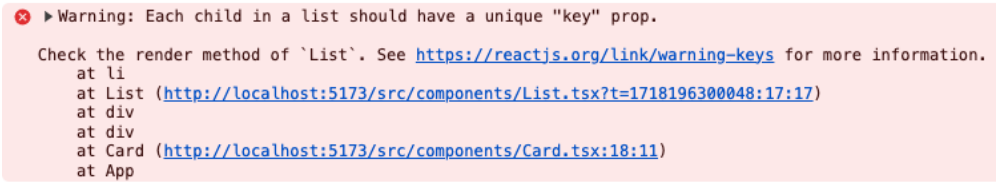
export default List;
```

Imprimir listas

Al hacer este mapeo, tenemos este resultado:



Pero también, se presenta un error en consola, el cual nos indica que cada uno de los hijos en un listado debe tener un identificador único, ya que generalmente estos datos vienen de una bd, y traen un id, pero como acá no estamos trabajando ninguna bd, debemos simular un identificador único “ key ”.



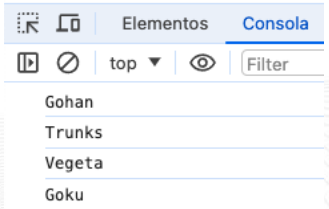

```jsx
<ul className="list-group">
  {data.map((elemento) => (
    <li key={elemento} className="list-group-item">{elemento}</li>
  ))}
</ul>
```

Gestión de eventos

Hemos creado una tabla mapeada, con los elementos que nosotros queremos, pero sigue siendo estática, así que seguimos trabajando en hacerla dinámica. Recordemos que a cada elemento HTML, se le puede agregar la función onClick, es por esto que vamos a asignarla con un console.log, para visualizar inicialmente elemento.

```
<li
  onClick={() => console.log(elemento)}
  key={elemento}
  className="list-group-item"
>
```

Con esto, si yo le doy click a cualquier elemento de la lista, este debería pintarse en la consola



Gestión de eventos

En este caso, la función flecha funciona dentro de onClick ya que es una sola línea, pero no es lo idea, ya que, si la función es muy grande, va a generar caos visual, así que lo que podemos hacer es sacar la función y crear una nueva función llamasa **“ handleClick ”** ¿Por qué se llama handleClick? Porque es una convención creada para indicar que va a ejecutar una acción, una vez se haga click sobre el elemento html.

```
type Props = {
  data: string[];
};

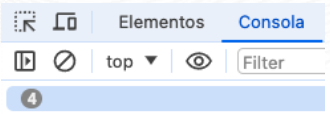
function List({ data }: Props) {
  const handleClick = () => {
    console.log("");
  };

  return (
    <ul className="list-group">
      {data.map((elemento) => (
        <li onClick={handleClick} key={elemento} className="list-group-item">
          {elemento}
        </li>
      ))}
    </ul>
  );
}

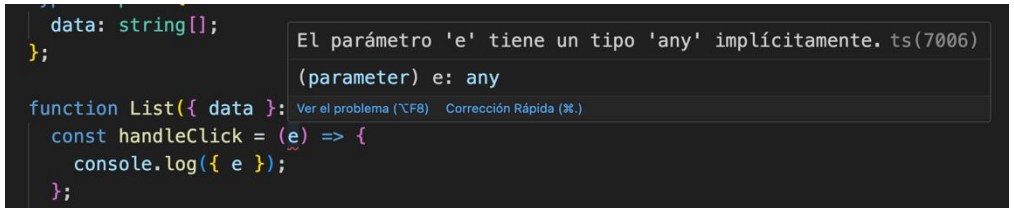
export default List;
```

Gestión de eventos

Como deje el console.log, vacío, miremos que pasa en la consola:



Cada que doy, click, efectivamente se muestra el vacio. Ahora, cada fucnion que le pasemos a onClick, va a recibir un evento, que en este caso sería un objeto que podemos pasar directamente en handleClick, por convención debería llamarse **event** pero typeScript, permite que se llame **e** pero al pasar el parámetro vemos un error:



Generalmente, esto es porque hay un MouseEvent, pendiente, por lo que le pasaremos este evento a e

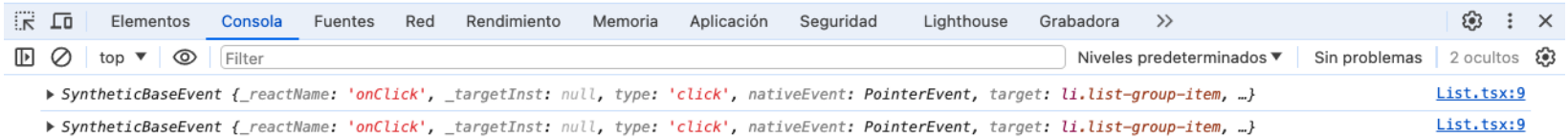
```tsx
import { MouseEvent } from "react";

type Props = {
  data: string[];
};

function List({ data }: Props) {
  const handleClick = (e: MouseEvent) => {
    console.log(e);
  };
```

Gestión de eventos

Listo, solucionamos el problema del evento, pero ahora veamos en consola que aparece



Tal como se observa, dice que tiene un evento sintetico, lo que significa que es el nombre del evento en react, que es una clase que se encarga de envolver los eventos dependiendo del explorador. Resumen, esta clase se encarga de envolver los elementos de cualquier navegador, para que nosotros podamos visualizar y que el resultado sea exactamente igual, en cualquier navegador que usemos.

Perfecto, con esto entendemos que vamos a poder trabajar en cualquier navegador, y react nos va a permitir visualizar lo mismo siempre, pero ya no quiero usar el MouseEvent, ya que no me da mucha información de mi elemento, por lo tanto, vamos a seguir, intentando hacer nuestra lista dinámica.

Gestión de eventos

Resulta que ahora, lo que quiero recibir es el elemento mismo que estamos iterando, y no el evento, asi que para esto, modifiquemos el onClick, así:

App.tsx | List.tsx | Card.tsx | Titulo.tsx

react-app > src > components > List.tsx > ...

```tsx
type Props = {
  data: string[];
};

function List({ data }: Props) {
  const handleClick = (e: string) => {
    console.log(e);
  };

  return (
    <ul className="list-group">
      {data.map((elemento) => (
        <li onClick={() => handleClick(elemento)} key={elemento} className="list-group-item">
          {elemento}
        </li>
      ))}
    </ul>
  );
}

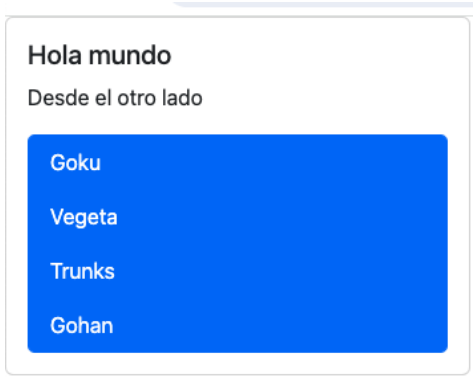
export default List;
```

¿Qué paso en la consola?

Gestión de estados

Con la gestión de eventos, lo que hacemos es indicar que cada acción que se ejecute tenga una respuesta, pero ahora la gestión de estados, buscamos que cada que pinchemos un elemento este se pinte como seleccionado, así que miremos la primera forma de hacerlo. Si en el claseName, ponemos active, veamos que pasa

```
className="list-group-item active"
```



Todos los elementos quedaron, seleccionados, así que por ahí es, pero claramente no es lo que necesitamos, por lo que debemos crear una variable que se encargue de hacer este trabajo, para mantener el índice del elemento que seleccionamos, asi que voy a crear una variable que se llame index

Gestión de estados

La variable por ahora no se va a inicializar con nada:

```
5   function List({ data }: Props) {
6       let index;
```

Luego, vamos a preguntar, si es que el índice del elemento que estamos iterando (.map) es exactamente el índice que se definió arriba, por lo que le damos un valor al índice

```
let index = 1;
```

Y le vamos a pasar el elemento i, a al mapeo, para que entienda que debe comparar

```
{data.map((elemento, i) => (
```

Para después en el className preguntar que si index, es igual al index del .map, escriba active así:

```
className={`list-group-item ${index == i ? 'active' : ''} `}
```

Cambie el valor de index, y observe que pasa en el navegador.

Gestión de estados

Como vimos, el estado se empieza a mover, pero no de la forma más dinámica posible, asi que lo que debemos hacer es usar una herramienta de react llamada **useState** la cual nos sirve para almacenar variables, pero que nosotros sepamos que pueden cambiar. Para empezar a usar, eliminemos la variable index, y reemplacemos por useState

```
import { useState } from "react";

type Props = {
  data: string[];
};

function List({ data }: Props) {
  useState(1);
```

Al usarlo, se debe tener la claridad que también se debe importar desde react, y que si quiero que no tenga ningún estado por defecto, le pongo -1, pero como no es el caso, le digo que inicie en el estado 1

Gestión de estados

Ahora, useState nos devuelve 2 valores, por lo que debemos inspeccionarlos y trabajarlos así:

```
const [index, setIndex] = useState(1);
```

Con esto, le indicamos que vamos a tener una variable index que no se va a modificar, para eso, usamos setIndex, la cual va a estar en constante cambio sin alterar por completo index. Ahora lo que debemos hacer modificar el handleClick teniendo en cuenta que vamos a pasar un index que es un número.

```
const handleClick = (i: number) => {
  setIndex(i);
};
```

Como cambie, el elemento por el index, este cambio también se debe hacer al momento de usar el onClick

```
onClick={() => handleClick(i)}
```

Ahora, en el navegador de click, sobre cualquier elemento de la lista y observe su comportamiento.

Gestión de estados

Ahora, lo que acabamos de ver es el manejo de estados haciendo uso de useState, pero useState, es un **Hook** de react, lo cual es una funcionalidad que nos permite modificar variables como componentes funcionales. En otras palabras

```
const [index, setIndex] = useState(1);
```

index = variable
setIndex = función

La función, nos permite actualizar el valor de la variable; y al ser actualizado, el funcionamiento interno de react, nos dice que cada que como cambio index, atreves de la función, lo que está pasando es que se re renderizara el componente completo nuevamente. Pero para cuando eso ocurra, el valor de useState, va a ser el nuevo valor que estamos enviando y es por esto, que nosotros podemos visualizar un cambio en pantalla.

¿Qué pasa si duplico List, en App.tsx? ¿Cómo hago para que no inicie ningún elemento seleccionado?

Funciones como props

Ya tenemos el componente de lista el cual recibe los componentes a renderizar y también muestra cuando cualquier elemento es seleccionado. Pero como lograría que pasara algo diferente como buscar datos en una API, como está actualmente ya que si vemos la lógica vemos que va a buscar los elementos, pero luego se tiene el problema que no vamos a poder usar estos elementos en otro componente ya que tiene toda la lógica en duro, es por esto por lo que debemos pasarle funciones para poder reutilizar el componente sin que muestre otra pantalla en otros componentes.

app > src > components > List.tsx > ...

```tsx
import { useState } from "react";

type Props = {
  data: string[];
};
function List({ data }: Props): JSX.Element
function List({ data }: Props) {
  const [index, setIndex] = useState(0);
  const handleClick = (i: number) => {
    setIndex(i);
  };
```

Funciones como props

Empecemos por pasarle a la función handleClick el elemento en onClick y obviamente para no tener problemas se lo pasamos también como string al momento de hacer su declaración:

```
const handleClick = (i: number, elemento: string) => {
  setIndex(i);
};

return (
  <ul className="list-group">
    {data.map((elemento, i) => (
      <li
        onClick={() => handleClick(i, elemento)}
        key={elemento}
```

Funciones como props

Seguidamente este elemento vamos a tener que usarlo, por lo que vamos a declarar una función llamada onselect, la cual va a recibir el elemento

```
const handleClick = (i: number, elemento: string) => {
  setIndex(i);
  onselect(elemento);
};
```

Vemos que nos muestra un error, y es porque no lo hemos definido en la función por defecto ni en el fragment de props que hacemos al inicio

```
type Props = {
  data: string[];
  onSelect: (elemento: string) => void;
};

function List({ data, onSelect }: Props) {
  const [index, setIndex] = useState(0);

  const handleClick = (i: number, elemento: string) => {
    setIndex(i);
    onSelect(elemento);
  };
```

Funciones como props

Sin embargo, esto puede traer problemas ya que habrá ocasiones en las que queramos pasarle una función y van a existite otras en las que no le pasemos nada, porque debemos indicar que se opcional

```
type Props = {
  data: string[];
  onSelect?: (elemento: string) => void;
};

function List({ data, onSelect }: Props) {
  const [index, setIndex] = useState(0);

  const handleClick = (i: number, elemento: string) => {
    setIndex(i);
    onSelect(elemento);
  };
```

Pero eso nos trajo otro problema en onSelect que nos dice que la función podría ser posiblemente no definida, por lo que para solucionar esto vamos a usar “ ?. ” lo que le indica que mire si la función esta definida, y sino no haga nada, de lo contrario ejecútela.

```
const handleClick = (i: number, elemento: string) => {
  setIndex(i);
  onSelect?.(elemento);
};
```

Funciones como props

Ahora, nuestro elemento está siendo usando App.tsx, si lo dejamos así, no hay lio ya que le dijimos que es una opción la función, pero vamos a pasarle propiedades directamente desde el App.tsx.
Recordemos que se va a ejecutar la función, por lo tanto, en onSelect, pasaremos un handleSelect, el cual se declara inmediatamente.

```
const handleSelect = (elemento: string) => {
  console.log(elemento);

}
return (
  <Card>
    <CardBody  title="Hola mundo" text="Desde el otro lado"/>
    <List data={list} onSelect={handleSelect}/>
  </Card>
);
```

Revisemos la consola del navegador.

State vs Props

State:

Datos internos de los componentes
El estado puede mutar

Props:

Argumentos
Inmutables

Cuando cambia alguno de los dos se ocasiona un RE-RENDER, lo que significa que al gestionar cualquiera de los dos, ese componente va a ser re renderizado.

Truthy y falsy en React

Cuando se trabaja JS, sabemos que hay valores que son True, o Falsa, a estos valores se les conoce como:

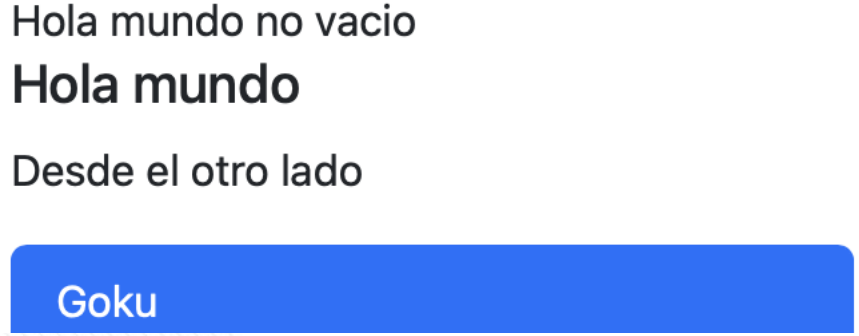
Truthy

Falsy = 0, “”, false, undefined, null

¿Porque es importante esto?

Porque si hacemos un condicional, cualquier valor de falsy nos da como resultado la parte negativa del condicional, sin embargo, en React esto NO ES ASI.
Si pongo un string vacio y comparo con un short circuit operator, tengo lo siguiente

```
10    return (
11      <Card>
12        {"" && "String vacio"}
13        {"Hola" && "Hola mundo no vacio"}
```



Truthy y falsy en React

Lo cual me indica que el string vació, no tiene relevancia, asi que probemos las demás opciones de false:

```
<Card>
  {"" && "String vacio"}
  {undefined && "String undefined"}
  {null && "String null"}
  {false && "String false"}
  {0 && "String 0"}
```

¿Cuáles se muestran en pantalla?

Truthy y falsy en React

Lo cual me indica que el string vació, no tiene relevancia, asi que probemos las demás opciones de false:



¿Cuáles se muestran en pantalla?

Truthy y falsy en React

Efectivamente el 0

> 0
> Hola mundo

Por lo que hay que tener mucho cuidado ya que habrá ocasiones en las que vamos a quere evaluar si es que existe un listado, y si existe vamos a querer mostrarlo, por ej:

```
return (
  <Card>
    {list.length && "Mi lista"}
```

Pero ahora, vamos a definir la lista como un array vacío y veamos.

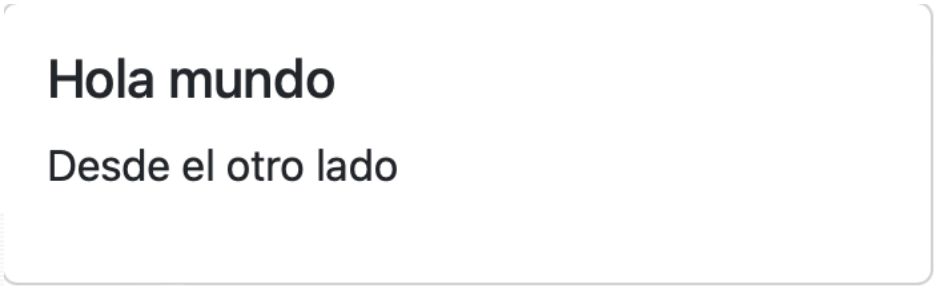
```
function App() {
  const list: string[] = [];

  const handleSelect = (elemento: string) => {
    console.log(elemento);
  };
  return (
    <Card>
      {list.length && "Mi lista"}
```

Truthy y falsy en React

Si nos fijamos, vuelve a aparecer el cero, y no es lo que quiero, pero como solo estamos evaluando la longitud de este array, al estar vacío, solo me devolverá ese valor en pantalla, por lo tanto, lo que se debe hacer es evaluar si es que list.length es distinto de cero así:

```
{list.length != 0 && "Mi lista"}
```



Renderizado condicional

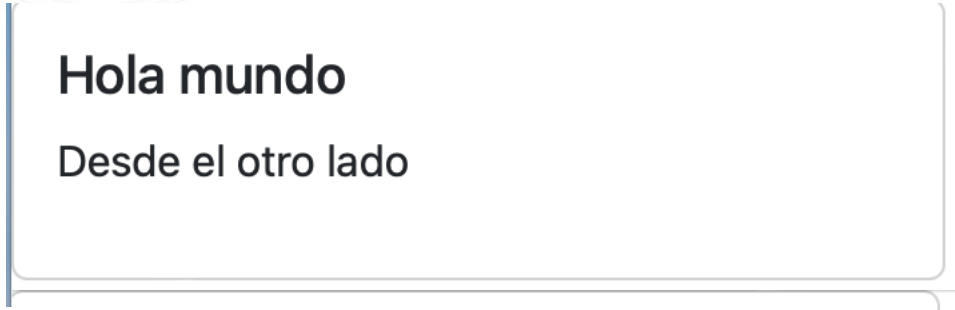
Con lo anterior forzamos la renderización de los componentes basando en si existe o no un elemento en la lista, ahora veamos cómo hacerlo de una forma un poco mejor.

Empecemos por crear una constante llamada contenido, la cual se va a encargar de guardar la lógica, es decir, que se renderiza y que no haciendo uso del operador ternario.

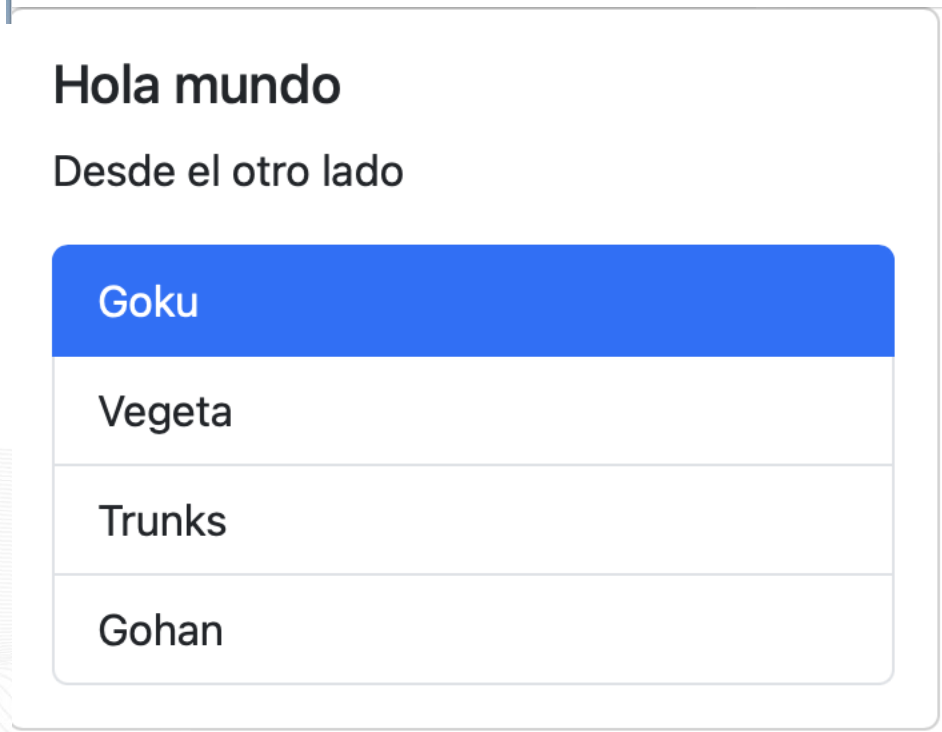
```
const contenido = list.length ? (
  <List data={list} onSelect={handleSelect} />
) : (
  "Sin elementos para mostrar"
);
return (
  <Card>
    <CardBody title="Hola mundo" text="Desde el otro lado" />
    <List data={list} onSelect={handleSelect} />
  </Card>
);
```

Renderizado condicional

Si revisamos el navegador nos encontramos con la lista vacía.



Pero si agregamos elementos:



Renderizado condicional

Recordemos que una buena práctica es usar el short circuit operator, y que este se usa justo antes del return, así que nuestro código debería verse algo como esto:

```
const contenido = list.length !== 0 && (
  <List data={list} onSelect={handleSelect} />
);
return (
```

Y para mostrar el contenido:

```
return (
  <Card>
    <CardBody title="Hola mundo" text="Desde el otro lado" />
    {contenido}
  </Card>
);
```

React devoloper tools

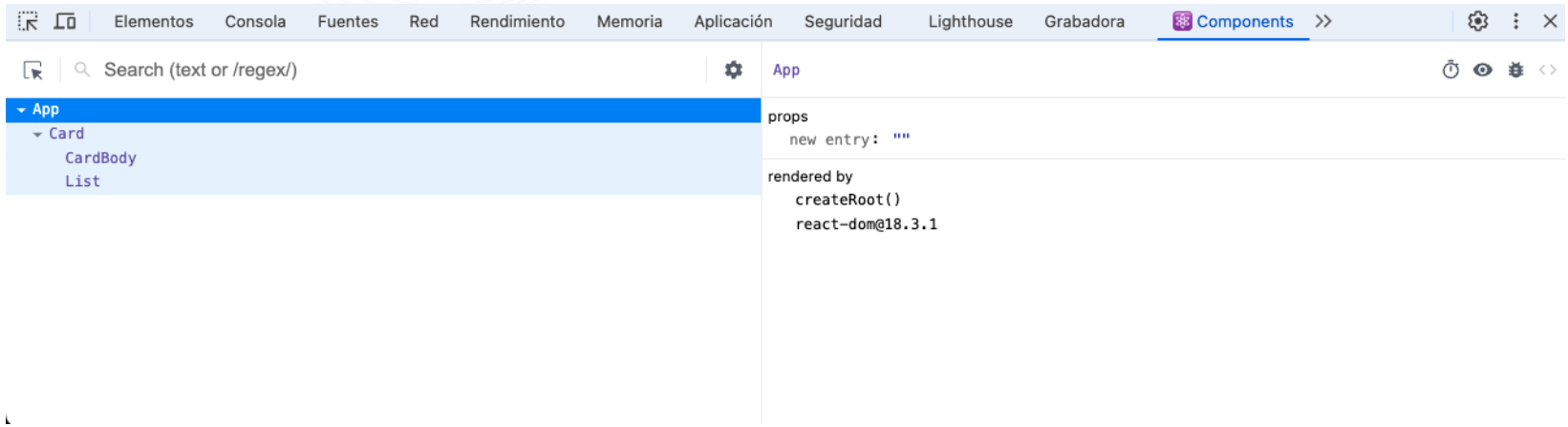
Veamos ahora una herramienta fundamental que ayuda a depurar las fuciones de una manera muy sencilla.
Se descarga de https://chromewebstore.google.com/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi para Google Chrome.
Ahora, para usarla vamos a abrir la consola



Y nos vamos a centrar en components:

React devoloper tools

Si nos fijamos, aca podemos ver todos los componentes que se están utilizando



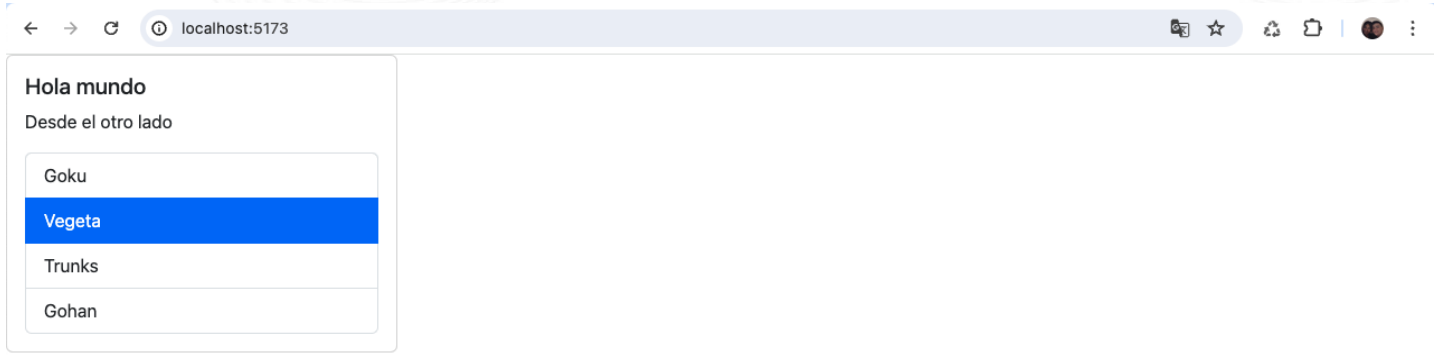
Ahora, si damos click en CardBody, podremos ver las propiedades que tenga en ese determinado momento

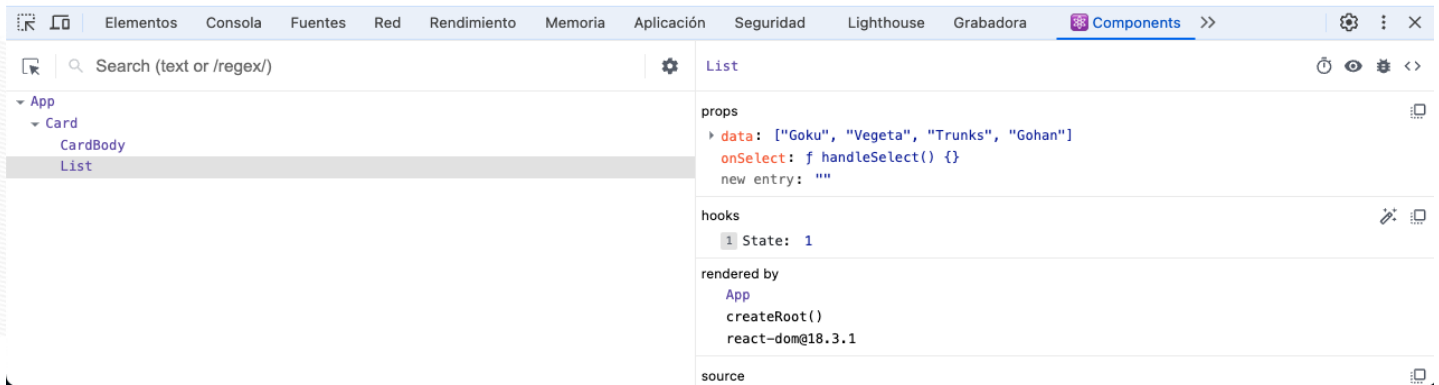


Y si bajamos más podremos ver, quien renderiza que en este caso es App y mas abajo en source, podremos ver el elemento especifico que lo esta importando

React devoloper tools

Analicemos un momento el componente List, ya que aparece hooks.





Practica

Debemos crear lo siguiente: Un botón que cambie de estado:



Y usar la herramienta de análisis de componentes, para poder cambiar su estado al inicial o final nuevamente.

OJO, se debe cambiar el estado, no la propiedad.

Practica

Lo primero que voy a hacer es crear el componente de Button e importarlo en app.tsx

```
<Button>
  Hola mundo
</Button>
```

```
import Button from "./components/Button";
```

Seguidamente, me dirijo a la creación del componente:

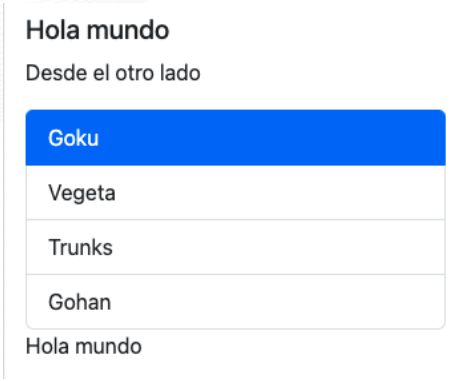
App.tsx 1 | Button.tsx | List.tsx | Card.tsx | Titulo.tsx

react-app > src > components > Button.tsx > default

```
1  import { ReactNode } from "react";
2
3  type Props = {
4      children: ReactNode;
5  }
6
7  function Button({children}: Props) {
8    return (
9      <div>{children}</div>
10   )
11 }
12
13 export default Button
```

Practica

Si recargo la página debería aparecer:



Ahora pasemos de Bootstrap la característica de botón.

```
function Button({children}: Props) {
  return (
    <button type="button" className="btn btn-primary">{children}</button>
  )
}
```

Practica

Ahora, el botón necesita un estado y una función que le permita cambiar, así que vamos a hacer uso del hook de useState asi;

```
function App() {

  const [isLoading, setIsLoading] = useState(false);
```

Ahora hay que crear una función que cuando toquemos el botón le pase su estado diferente a la inicial.

```
const [isLoading, setIsLoading] = useState(false);
const handleClick = () => setIsLoading(!isLoading);
```

Ahora pasemos la función al botón así:

```
<Button isLoading={isLoading}  onClick= {handleClick}>Hola mundo</Button>
```

Nada debería funcionar, por lo que editemos el componente Button

Practica

En propiedades, vamos a añadir lo que acabamos de crear:

```
type Props = {
    children: ReactNode;
    isLoading: boolean;
    onClick: () => void;
}
```

Ahora, tendremos que hacer el destructuring:

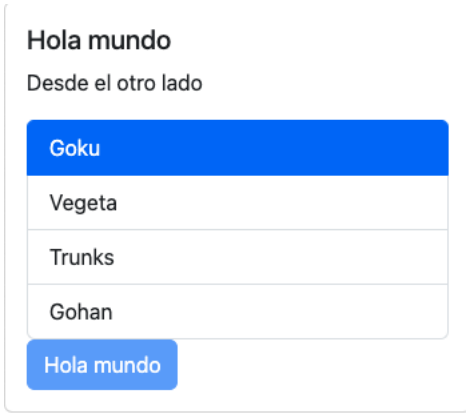
```
function Button({children, isLoading, onClick}: Props) {
```

Y al botón indicarle si este se encuentra deshabilitado o no.

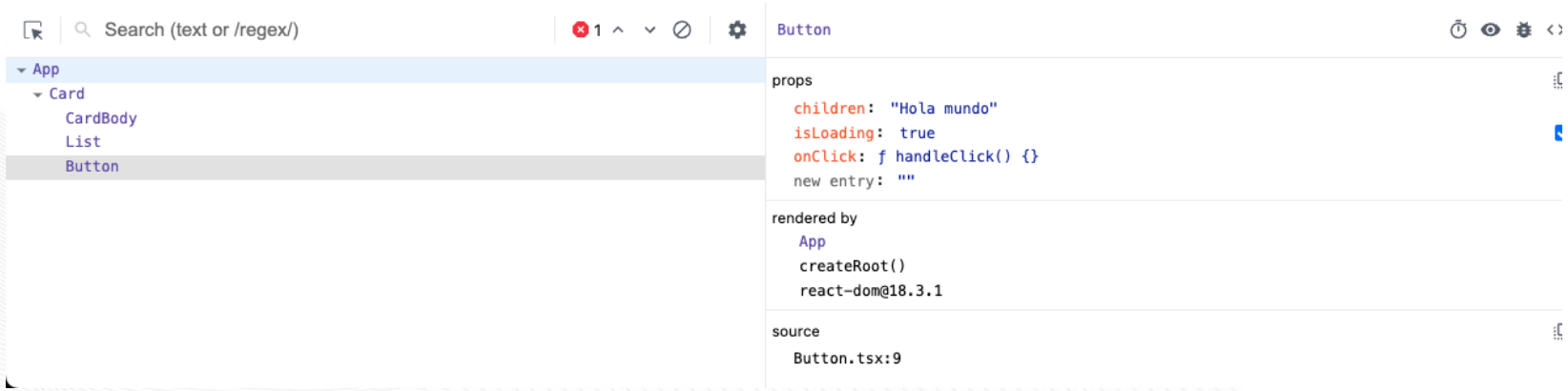
```
<button onClick={onClick} disabled={isLoading} type="button" className="btn btn-primary">
  {children}
</button>
```

Practica

Veamos en el navegador que sucede si pinchamos el botón:

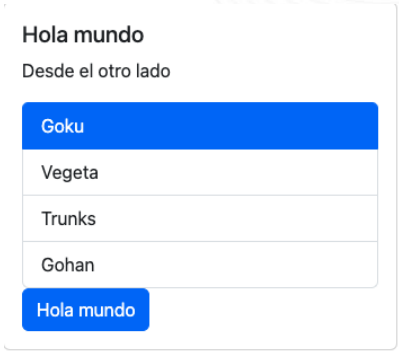


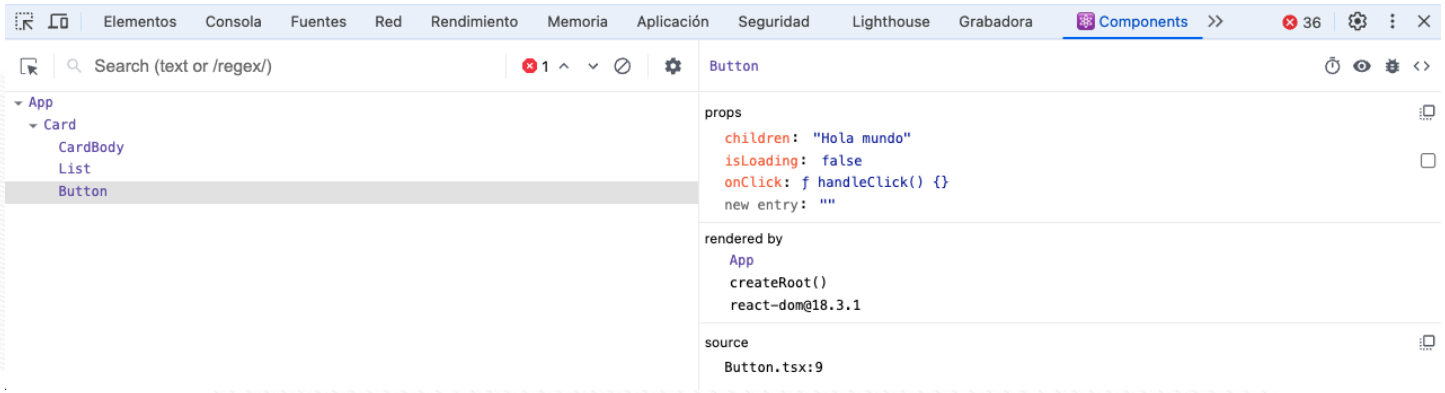
Aparece desabilitado, y si revisamos, efectivamente el estado se encuentra en true para el disable.



Practica

De click en true, y cambie a false, y observe el resultado.





Practica

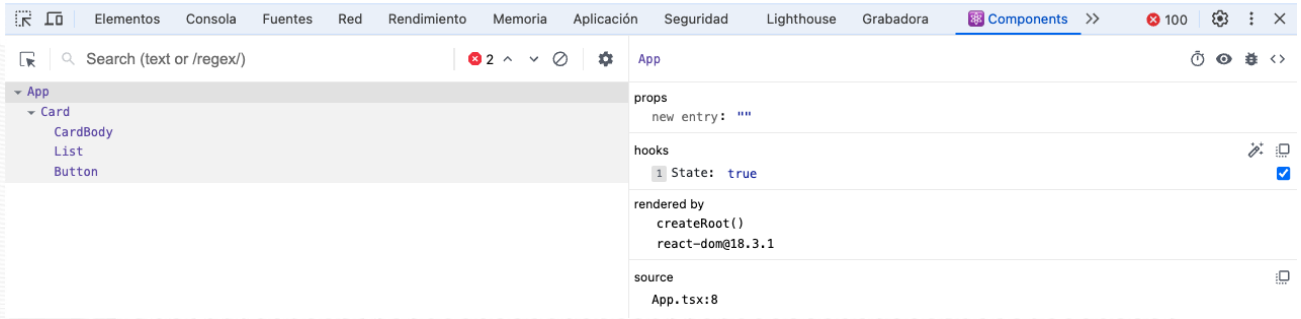
Ahora vamos a cambiar el texto para que diga cargando y también cambie el estilo del botón.
Para esto en el children preguntaremos que, si está cargando muestre el primary, si no, que cambie asi:

```
function Button({ children, isLoading, onClick }: Props) {
  return (
    <button
      onClick={onClick}
      disabled={isLoading}
      type="button"
      className={`btn btn-${isLoading ? "secondary" : "primary"}`}
    >
      {isLoading ? "Cargando..." : children}
    </button>
  );
}
```

Practica

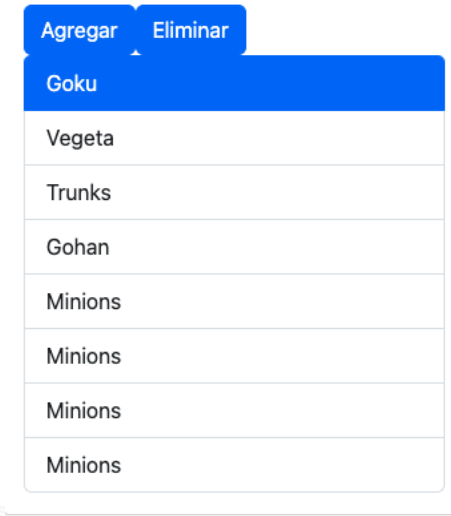
Ahora, lo que falta es cambiar el estado, y sé que van a querer cambiar desde botón pero no es ahí donde se debe hacer, se debe cambiar desde App.

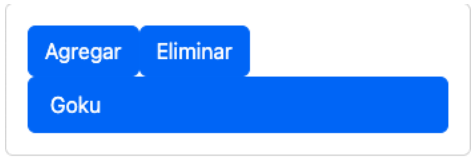




Practica

Sigamos poniendo en práctica, ahora quiero que hagan el clásico ejemplo de agregar y eliminar elementos a una lista, así que van a crear dos botones al inicio de la lista que agreguen y eliminen la palabra minion.





Practica

Agregar minios:

```
const [data, setData] = useState(["Goku", "Vegeta", "Trunks", "Gohan"]);
const addMinion = () => {
  setData([...data, "Minions"]);
}
return (
  <Card>
    <Button onClick={addMinion}>Agregar </Button>
    <Button>Eliminar </Button>
    <List data={data} />

  </Card>
);
```

Practica

Eliminar minios:

```
const [data, setData] = useState(["Goku", "Vegeta", "Trunks", "Gohan"]);
const addMinion = () => {
  setData([...data, "Minions"]);
}

const delMinions = () => {
  setData(data.slice(0, -1));
  //toma el array desde el inicio y quita el ultimo elemento del array
}
return (
  <Card>
    <Button onClick={addMinion}>Agregar </Button>
    <Button onClick={delMinions}>Eliminar </Button>
    <List data={data} />

  </Card>
);
}
export default App;
```

GRACIAS

Línea de atención al ciudadano: 01 8000 910270
Línea de atención al empresario: 01 8000 910682

@SENAComunica

www.sena.edu.co